LAB: 1 Signed Addition and Subtraction Using Two's Complement

Objective

To understand two's complement signed number representation and perform signed addition and subtraction with identification of overflow and non-overflow conditions.

Theory:

Introduction

Computers must handle both positive and negative numbers.
To do this efficiently, computers use two's complement representation, which allow:

Addition and subtraction using the same hardware

Simple overflow detection

No separate subtraction circuit in ALU

3. Number Representation

3.1 Signed Number Range

For an n-bit system:

$$Range = -2^{(n-1)} \text{ to } +2^{(n-1)} - 1$$

For 8-bit systems:

Minimum = −128

Maximum = +127

3.2 Two's Complement Formation

To find the two's complement of a number:

Convert magnitude to binary

Take 1's complement

Add 1

Example:

+5 = 00000101

1's = 11111010

+1 = 11111011 → −5

4. Signed Addition

4.1 Concept

Signed addition is performed by:

Adding two numbers in two's complement

Ignoring carry out of MSB

Checking overflow using sign bits

4.2 Overflow Rule (VERY IMPORTANT)

Overflow occurs ONLY IF:

Positive + Positive → Negative

Negative + Negative → Positive

👉 Overflow does NOT depend on carry

4.3 Algorithm: Signed Addition

Step 1: Start
Step 2: Read signed numbers `A` and `B`
Step 3: Convert `A` and `B` into n-bit two's complement
Step 4: Add the binary numbers
Step 5: Ignore carry out of MSB
Step 6: Check overflow:
If both inputs have same sign and result has opposite sign → Overflow
Step 7: Display result
Step 8: Stop

4.5Flowchart

4.5Numerical Examples (Addition)

☑ Example 1: No Overflow

Add: (+20) + (−5)

+20 = 00010100

−5 = 11111011

Sum = 00001111 = +15

✓Different signs → No Overflow

Example 2: Overflow

Add: (+70) + (+80)

+70 = 01000110

+80 = 01010000

Sum = 10010110 (negative!)

✗ *Two positives give negative → Overflow*

5. Signed Subtraction

5.1 Concept

Subtraction is performed as:

$$A - B = A + (\text{two's complement of } B)$$

Thus, subtraction is addition in disguise.

5.2 Overflow Rule (Subtraction)

Overflow occurs when:

Positive − Negative → Negative

Negative − Positive → Positive

5.3 Algorithm: Signed Subtraction

Step 1: Start
Step 2: Read signed numbers `A` *and* `B`
Step 3: Find two's complement of `B`
Step 4: Add it to `A`
Step 5: Ignore carry out
Step 6: Check overflow:
If result sign is incorrect → Overflow
Step 7: Display result
Step 8: Stop

5.4Flowchart

5.5 Numerical Examples (Subtraction)

☑ *Example 3: No Overflow*

Subtract: (+30) − (+10)

+30 = 00011110

+10 = 00001010

2's complement of +10 = 11110110

Result = 00010100 = +20

✓*Valid → No overflow*

Example 4: Overflow

Subtract: (+50) − (−80)

```
+50 = 00110010
-80 = 10110000
2's complement of -80 = 01010000
----------
Result = 10000010 (negative!)
```

✗ *Positive − Negative ⟶ Negative*

➡ *Overflow*

6.Relation to ALU (Hardware Insight)

ALU uses only adders

Subtraction = addition with inverted B and carry-in = 1

Overflow is detected using:

$$Overflow = Carry\ into\ MSB \oplus Carry\ out\ of\ MSB$$

7. Implementation in MATLAB

```
% ----------------------------------------------------
Signed Addition and Subtraction
% ----------------------------------------------------
```

```
clc;            % Clear command window
clear;          % Clear workspace

% ---------------- INPUT SECTION ----------------

A = input ('Enter first signed number A: ');  % Input first signed integer
B = input('Enter second signed number B: '); % Input second signed integer

n = 8;          % Number of bits (8-bit signed system)

% ---------------- TWO'S COMPLEMENT REPRESENTATION ----------------

A_bin = dec2bin(mod(A, 2^n), n);   % Convert A to n-bit two's complement
B_bin = dec2bin(mod(B, 2^n), n);   % Convert B to n-bit two's complement

disp(' ');
disp('--- INPUT VALUES ---');
disp(['A (Decimal): ', num2str(A), '   A (Binary): ', A_bin]);
disp(['B (Decimal): ', num2str(B), '   B (Binary): ', B_bin]);

% ---------------- SIGNED ADDITION ----------------

Sum = A + B;                            % Perform signed addition
Sum_bin = dec2bin(mod(Sum, 2^n), n);          % Convert sum to binary

% Overflow detection for signed addition
if (A >= 0 && B >= 0 && Sum < 0) || (A < 0 && B < 0 && Sum >= 0)
   add_overflow = 'YES';
else
   add_overflow = 'NO';
end
```

```
% ---------------- SIGNED SUBTRACTION ----------------

Diff = A - B;                          % Perform signed subtraction
Diff_bin = dec2bin(mod(Diff, 2^n), n);        % Convert difference to binary

% Overflow detection for signed subtraction
if (A >= 0 && B < 0 && Diff < 0) || (A < 0 && B >= 0 && Diff >= 0)
    sub_overflow = 'YES';
else
    sub_overflow = 'NO';
end

% ---------------- OUTPUT SECTION ----------------

disp(' ');
disp('--- SIGNED ADDITION RESULT ---');
disp(['Sum (Decimal) : ', num2str(Sum)]);
disp(['Sum (Binary)  : ', Sum_bin]);
disp(['Overflow      : ', add_overflow]);

disp(' ');
disp('--- SIGNED SUBTRACTION RESULT ---');
disp(['Difference (Decimal) : ', num2str(Diff)]);
disp(['Difference (Binary)  : ', Diff_bin]);
disp(['Overflow             : ', sub_overflow]);

% ---------------- END OF PROGRAM ----------------
```

8. Result

9.Discussion/Conclusion